Embedded Controller

Memory Mapped I/O

Young-Keun Kim
School of Mechanical & Control Engineering

Last modified: 2025.8

Introduction

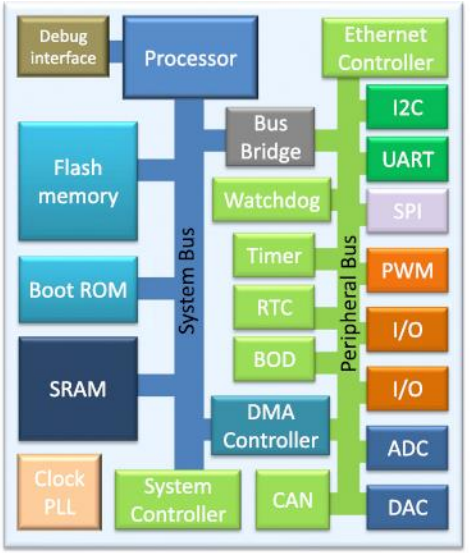
- **Part 1**
  - Memory Introduction
  - ROM, RAM
  - Register in CPU

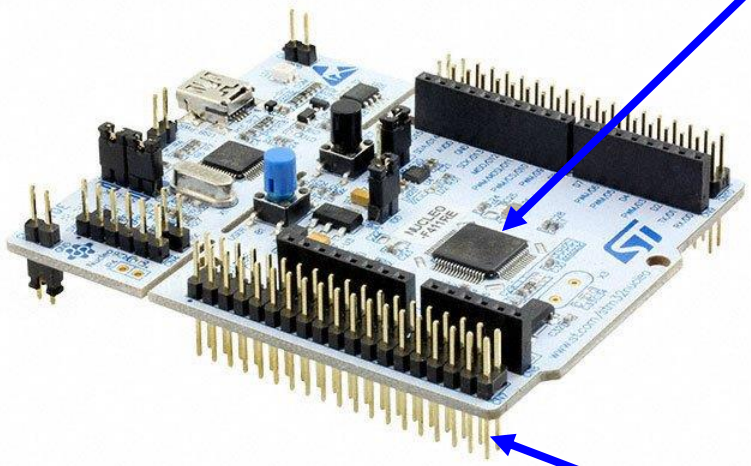
- **Part 2**
  - Memory Map in MCU
  - Memory Mapped I/O
  - DMA*

Introduction

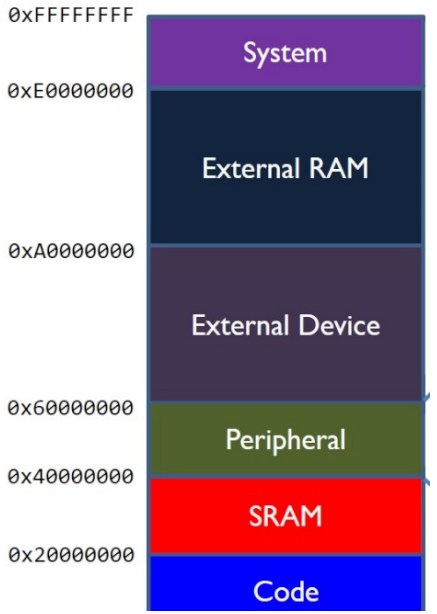
- Memory in MCU







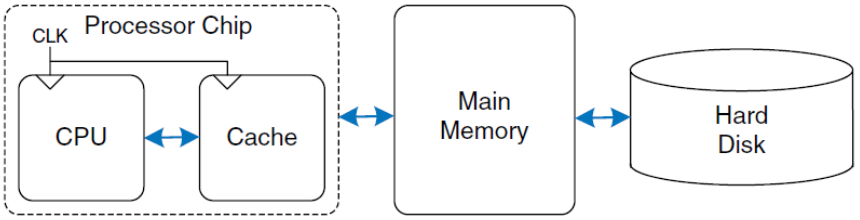
Peripheral & GPIO
Hardware pins

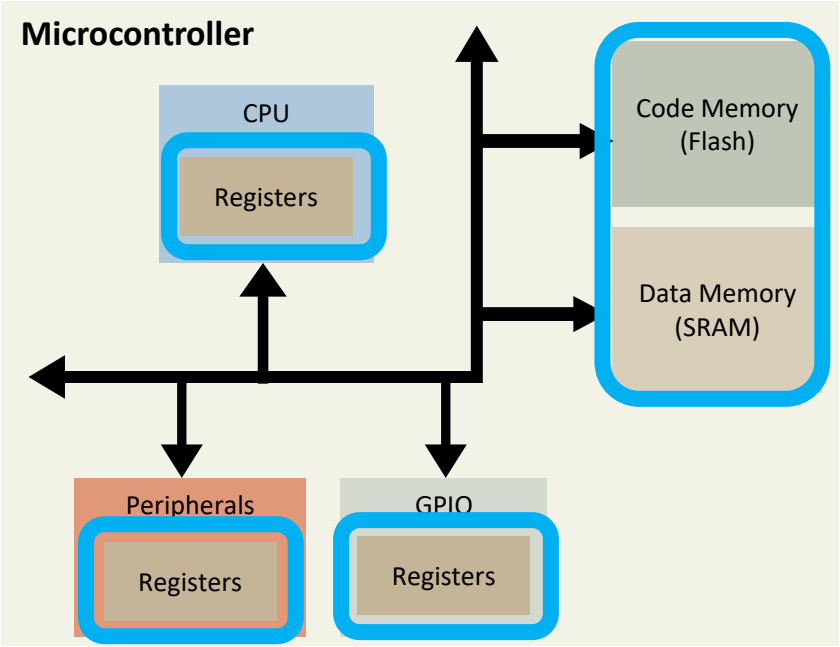


Introduction

- **Memory in MCU**
  - Mainly divided as
    - Code Memory (Flash/ROM)
    - Data Memory (SRAM)
    - **Register** (CPU, Peripherals)
    - System Memory
    - External RAM/SDCard etc

Memory in PC



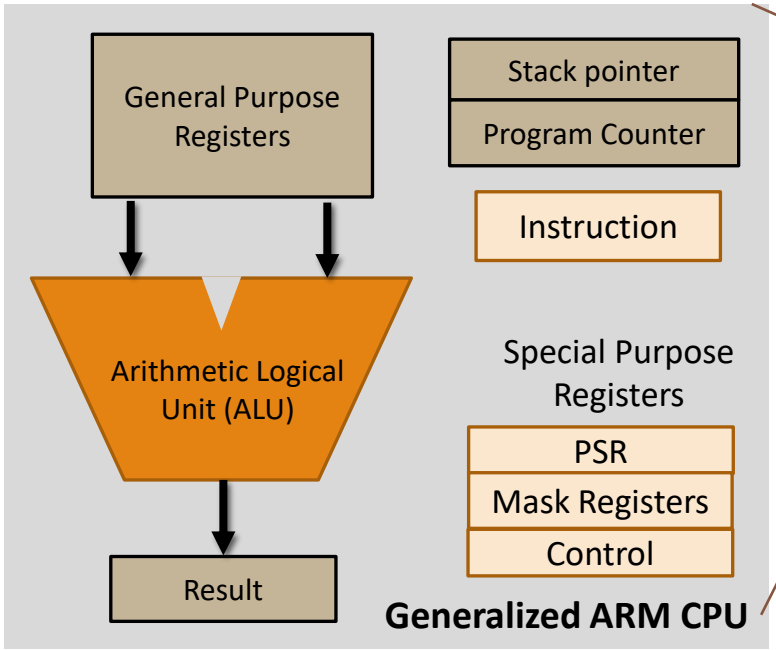


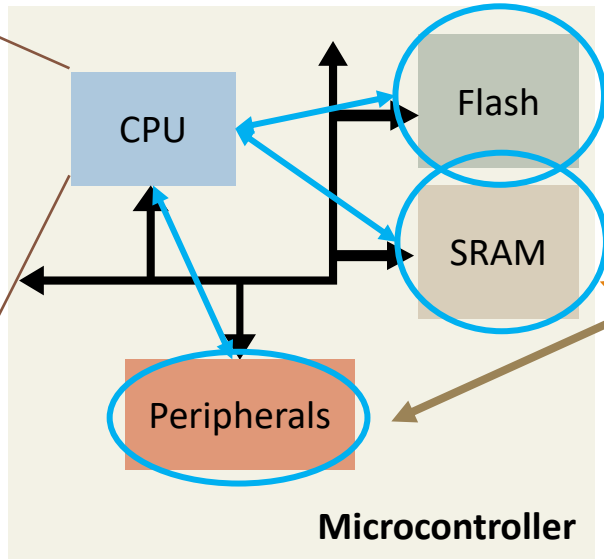
Interacting Memory [Coursera]

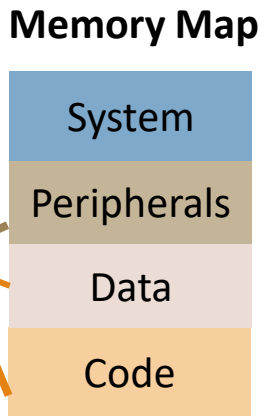
한동대학교 HANDONG GLOBAL UNIVERSITY

Introduction

- **Registers**
  - Fast, small memory for specific purpose
  - **CPU: Processor Registers**
    - General-Purpose Register: R0~R15 for ARM
    - Special Purpose: 5
  - **Peripherals/GPIO: Special Function Register**
    - Directly control I/O hardware

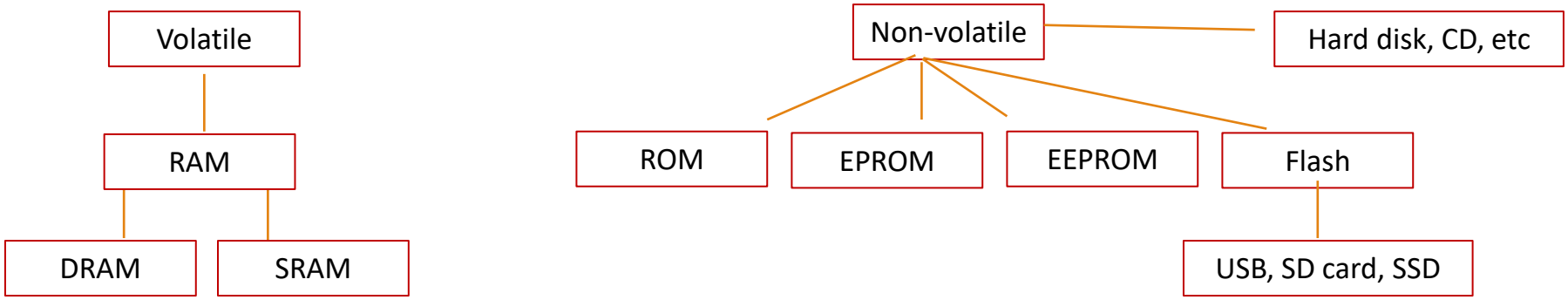






Types of Memory

- **What are the memory types?**
  - Basic types of Memory
    - Non-Volatile: ROM, EPROM, FLASH
    - Volatile: RAM(DRAM, SRAM)



DRAM: Transistor & capacitor, need to refresh due capacitor leakage
SRAM: Uses inverters, no refresh. Less memory per area, more expensive, faster
F-F: Fastest, more transistor per area than others

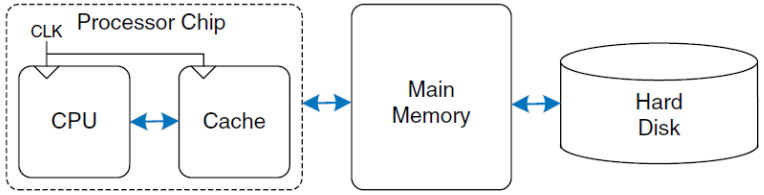
한동대학교 HANDONG GLOBAL UNIVERSITY

ROM vs RAM

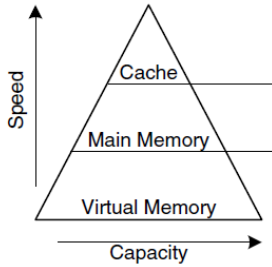
- **ROM and RAM**
  - ROM: Read Only Memory
  - RAM: Random Access Memory

  - Names are misleading for ROM
    - It is also random access. Some types of ROM can be written.

  - ROM and RAM are distinguished by
    - Non-volatile: ROM, EPROM, FLASH
    - Volatile: RAM(DRAM, SRAM)



- What to remember for now
  - MCU Instructions are stored in ROM (Flash)
  - Data are stored in SRAM
  - Peripheral are configured by Register



Technology	Cost/GB	Access Time
SRAM	~ $10,000	~ 1 ns
DRAM	~ $100	~ 100 ns
Hard Disk	~ $1	~ 10,000,000 ns

한동대학교 HANDONG GLOBAL UNIVERSITY

ROM

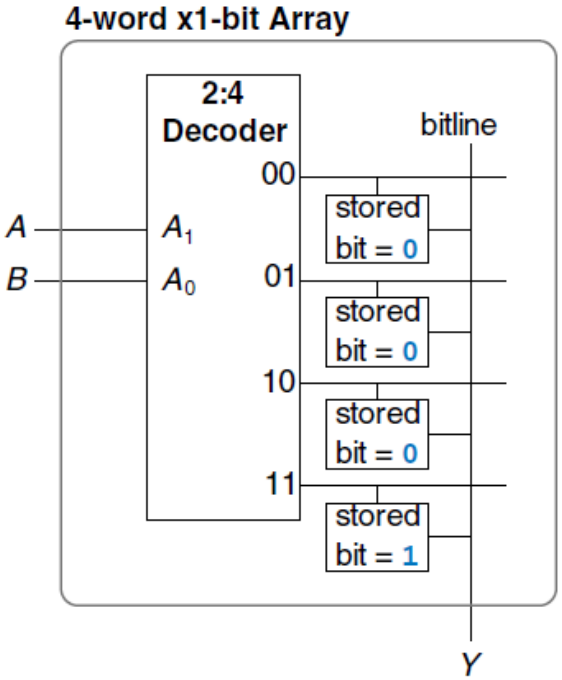
- **ROM (read only memory)**
  - Data that normally can only be read
  - ROM method of storing truth table: **LUT method**
  - Data in ROM is not lost when the computer power is turned off
  - ROM bit cell is a combinational circuit, no state to "forget" if power is turned off.

- **MCU Instructions are stored in ROM(Flash)**

- Certain types of ROM can be re-programmed
  - MASK ROM (x)
  - PROM (once)
  - EPROM (by UV light)
  - EEPROM (electronically, about 100k times)
  - Flash

Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



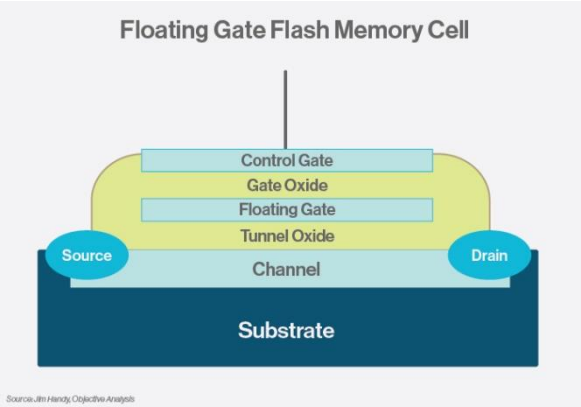
한동대학교 HANDONG GLOBAL UNIVERSITY

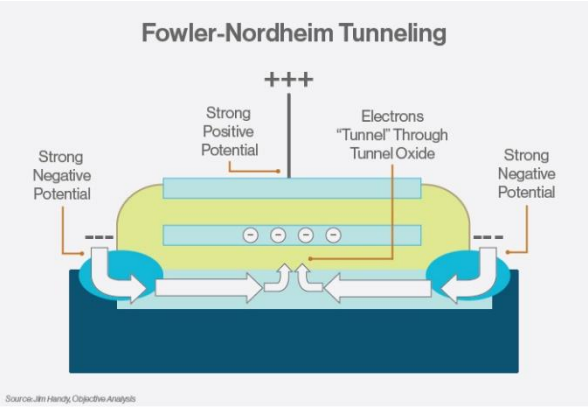
Flash Memory

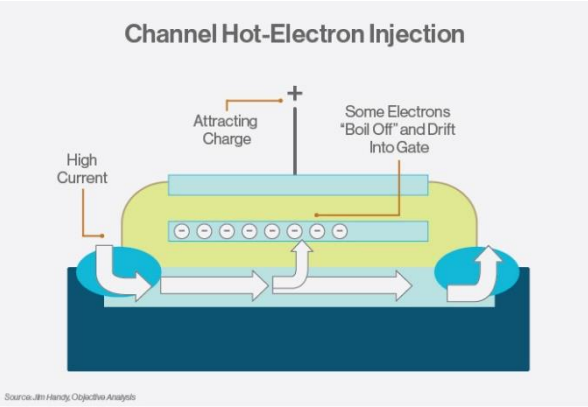
- **Flash Memory** (SSD, SD Card, USB memory, etc)
  - Positioned between ROM and RAM, can be read and written
  - Unlike SRAM and DRAM, flash memory is nonvolatile, offers lower power consumption
  - Slower in write than read mode. Slower than RAM
  - Has limited program/erase cycles

▪ How Flash Memory Works

*Invented in in 1970s by Toshiba, Commercialized by Intel in 1988

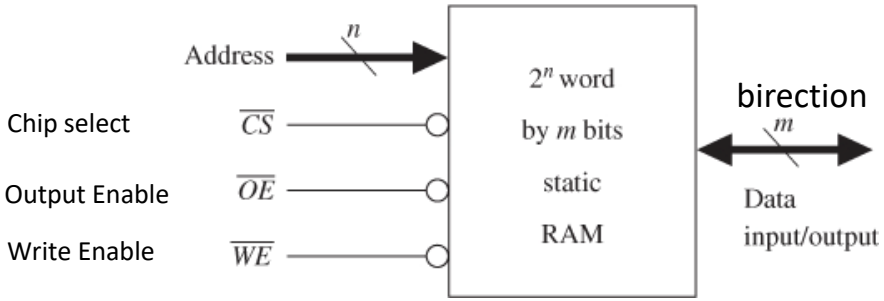






RAM

- **RAM (Random Access Memory)**
  - A type of memory that can be accessed randomly
  - Commonly used for main memory of computers, and other devices
  - Data in RAM is lost when the computer power is turned off
  - Types of RAM
    - **SRAM**
    - DRAM
    - High Speed DRAM

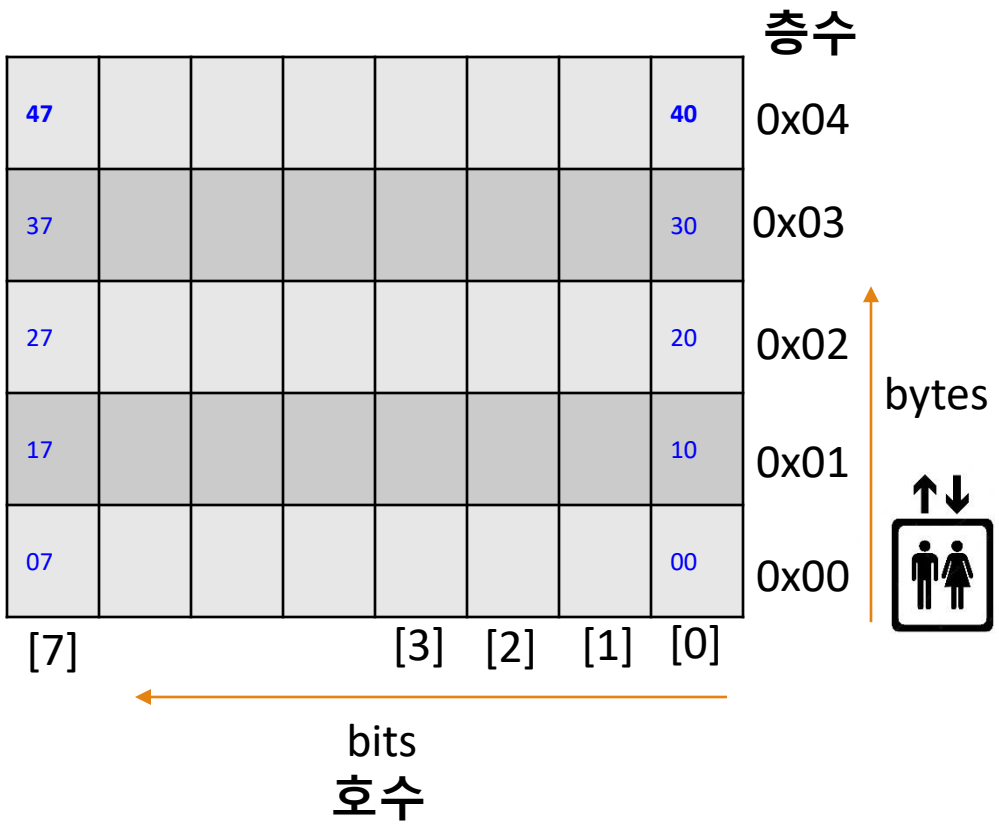


~CS	~OE	~WE	Mode	I/O pins
H	X	X	Not selected	high-Z
L	H	H	Output disabled	high-Z
L	L	H	Read	Data out
L	X	L	Write	Data in

한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Address

- **Memory Address**
  - Memory address is in **bytes**, *not bits*
    - 1 byte= 8-bits



한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Address

- **Memory Address**
  - Memory address is in **bytes**, *not bits*
  - Up to 2^m address(bytes) by m bits
  - Word Size
    - 32-bit MCU: 32 bits for 1 word
      → each word is 4 bytes=32 bits data
    - 8-bit MCU: (8+8)16 bits for 1 word
      → each word is 2 bytes=16 bits data

32-bit MCU (ARM Cortex-M)
32-bit→ 2^32 bytes =4 giga bytes
1 word: 4 bytes (4*8=32 bits) of width

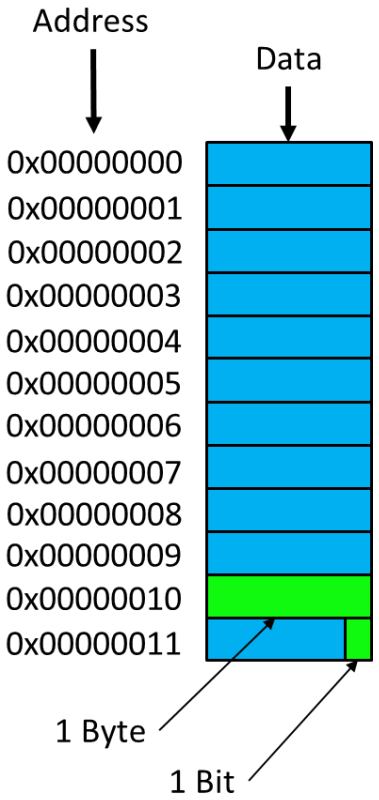
8-bit MCU (Arduino)
(8+8)-bit→ 2^16bytes =65 kilo bytes
1 word: 2 bytes (16 bits) of width

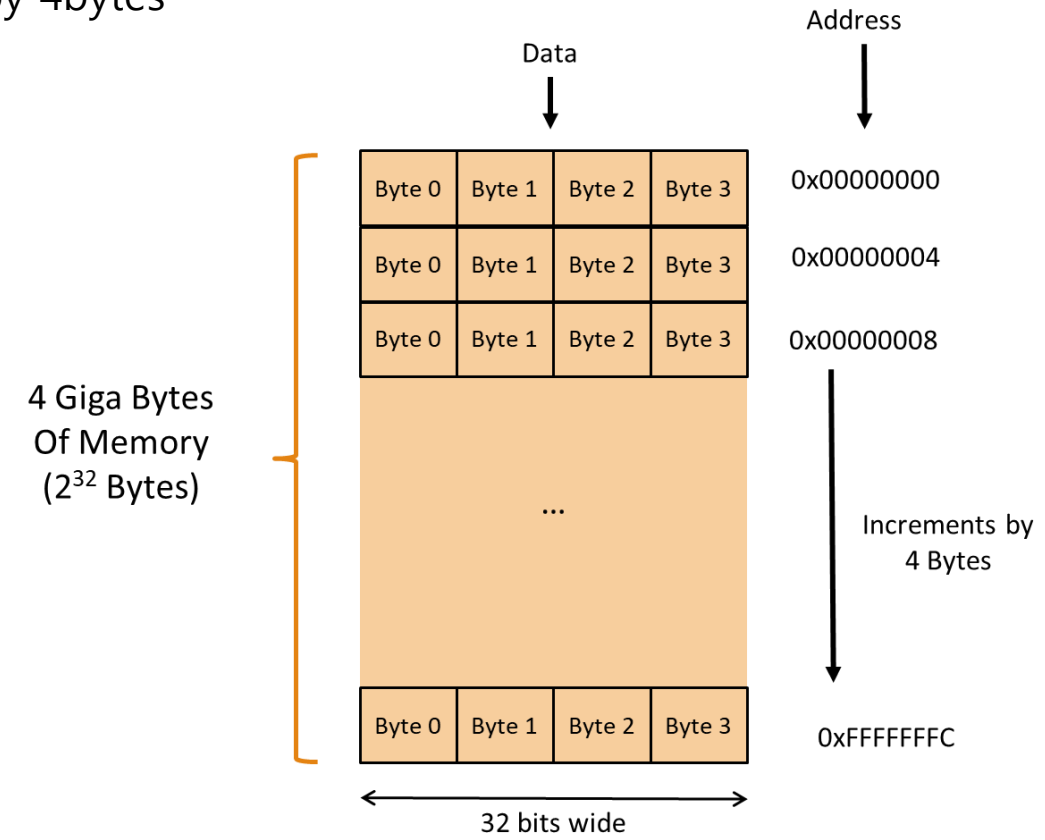
Microcontroller	Arduino	SRAM (bytes)
Atmega 328, Atmega 328p	Uno Rev3 SMD, Uno Wifi Rev2, Duemilanove, Nano, Bluetooth, Fio	2K
Atmega 32u4	Micro, Yun, Leonardo, Flora	2.5K
Atmega 2560	Mega	8K
32 bit ARM Cortex M3	SAM Boards: Due	32-100K
32 bit ARM Cortex M0+	SAMD Boards: Nano 33 IoT, MKR Fox 1200, Tian.	32K
nRF52	Nano 33 BLE, Sense.	256K

https://www.circuitbasics.com/types-of-memory-on-the-arduino/

Memory Address

- **32-bit MCU**
  - Up to 4-Giga Bytes(2^32) of memory
    - 1 word: 4 bytes (4*8=32 bits) of width
  - 4 bytes are stacked as one word
  - Each memory block is increases by 4bytes



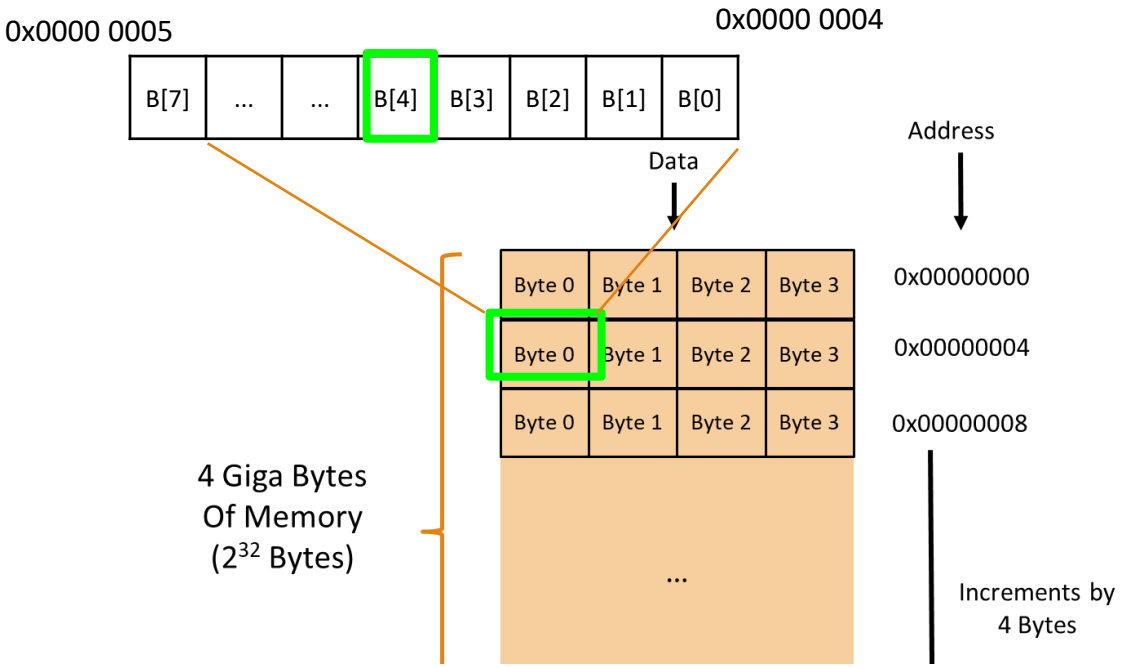


한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Address

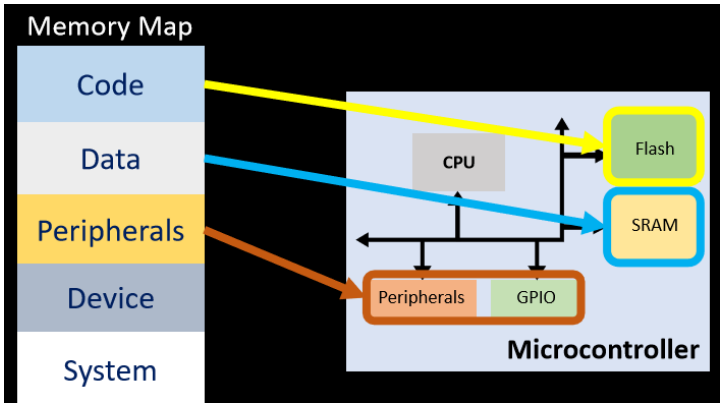
- **32-bit MCU**
  - If we can point an address in **Byte** unit, then, how can we read a certain bit in that byte?
    - e.g. Accessing BITS[4] of the data BITS located @ 0x0000 0004 byte

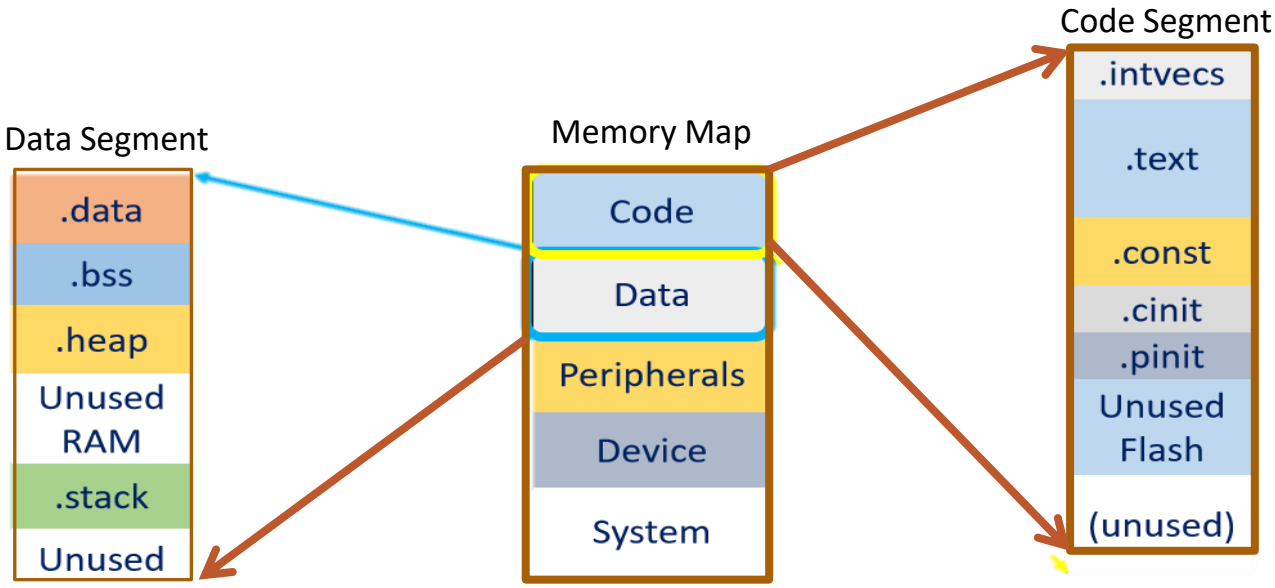
  - First, point to address 0x0000 0004 // BITS starting address
  - Then, use bitwise operation to access // bool readBit=BITS & (1<<4))
    // bool readBit=(BITS>>4) & 1



Memory Map

- **MCU Memory Map**
  - There are different memory types, different purposes
    - Memory for storing machine code
    - Memory for storing data (heap,stack,..)
    - **Memory for Peripheral I/O**



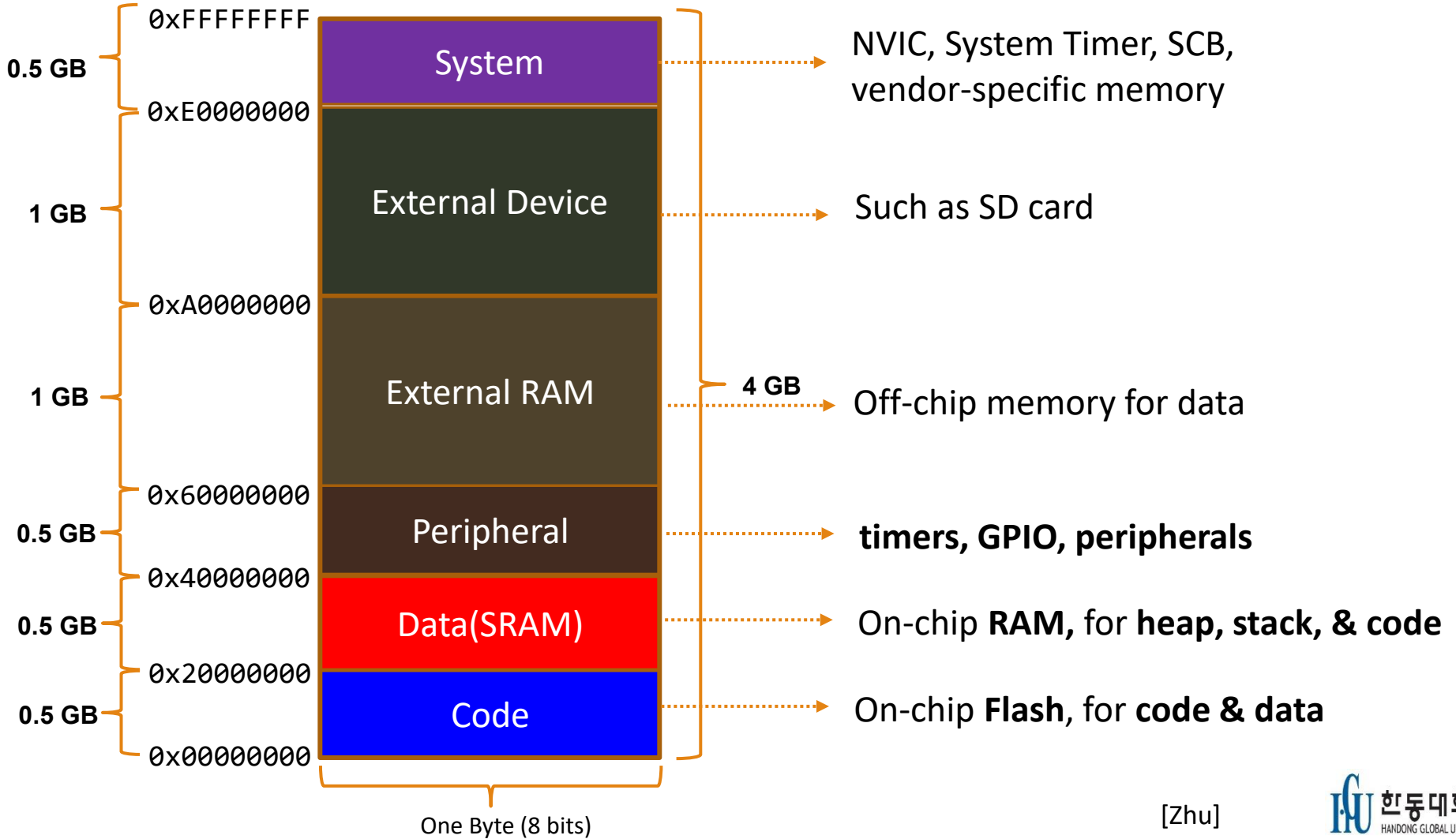


한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Map

- **Memory Map of 32-bit MCU**
  - Supporting 4 gigabytes of memory space
  - 8 hexa digits: 4bits/hexa*8hexa=32 bits

up to 0xFFFF FFFF (4.3×10^6)



[Zhu]

한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Map

- **Code Segment**

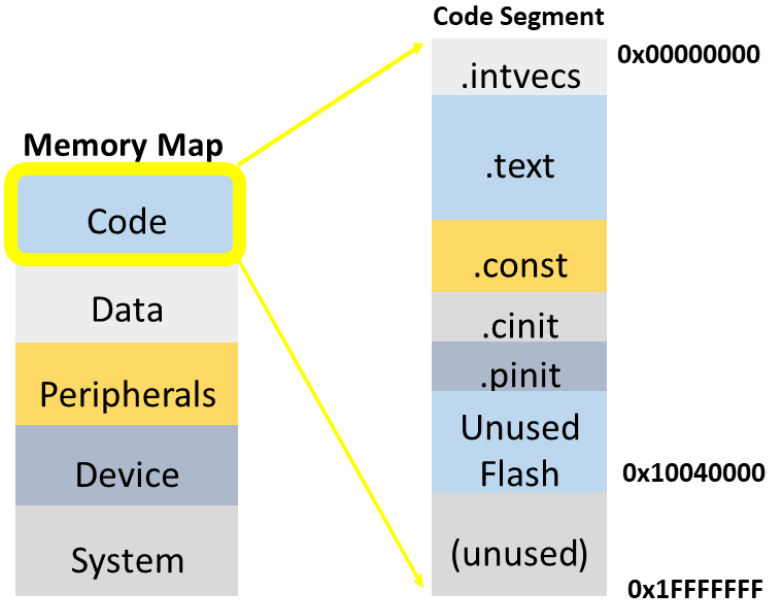
  stores instructions and some data in Flash

  - **intvecs**
    - Vector Tables contains platform defined list of function address

  - **Text**
    - Software instructions(machine code)
    - Main()
    - Standard Library Code()
    - User defined Functions()

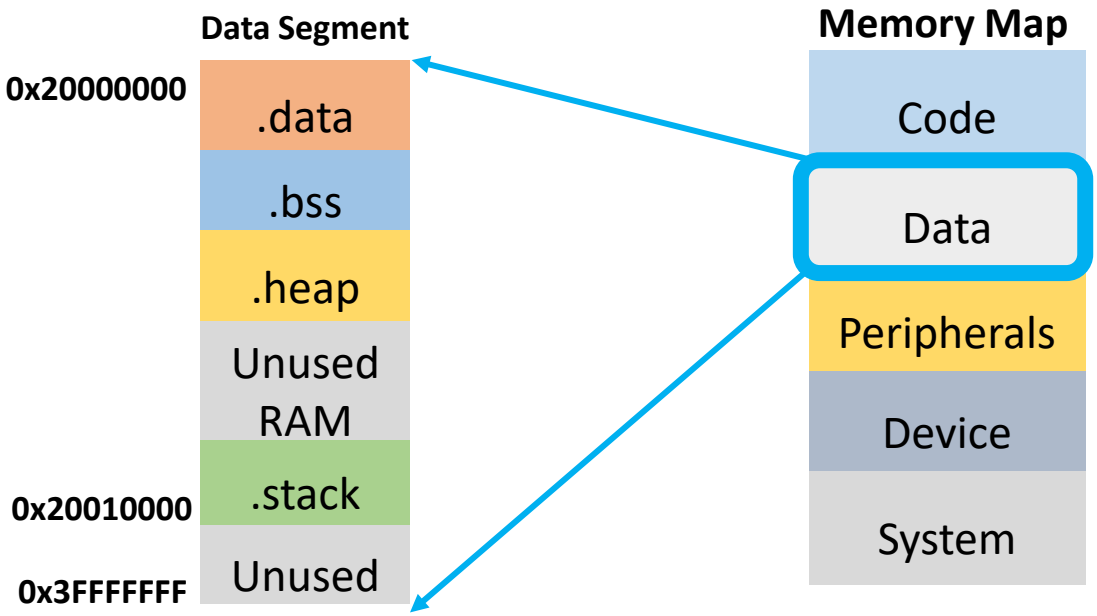
  - **const**
    - constant variable defined data
    - Read-Only (RO)

  - **pcint/cint**
    - Code used to initialize software or data



Memory Map

- **Data Segment**
  - Container for the various types of allocated data
  - Sub-Segments
    - **Stack:** Temporary Data Storage like local variables
    - **Heap:** Dynamic data storage
    - **Data:** Non-Zero Initialized global and static data
    - **BSS** (Block Started by Symbol) : Zero initialized and Uninitialized global and static data



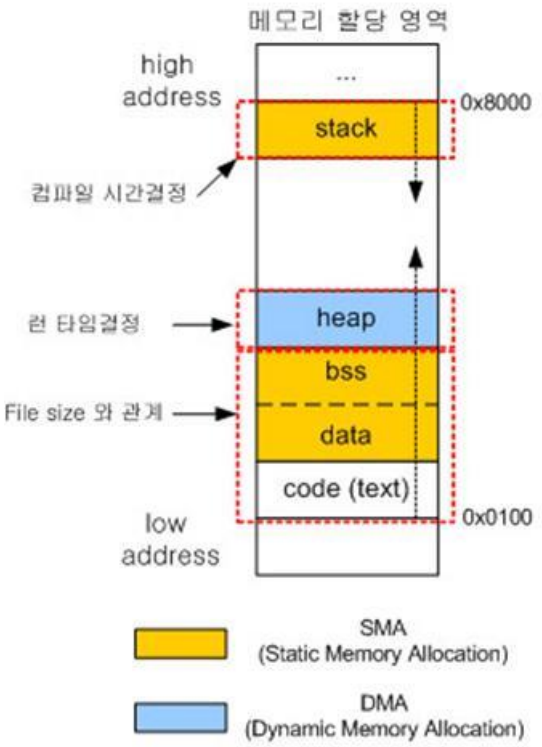
한동대학교 HANDONG GLOBAL UNIVERSITY

Memory Map

- **Data Segment**
  - **Stack:** size is determined by the compiler and freed automatically at the end of program
  - **Heap**
    - Memory is determined at run-time. User can dynamically allocate the size of variables
    - Should be freed to avoid memory fragmentation and other issues
    - Check the size of the heap of the RAM.
    - De-allocate('free') memory when it is not needed.

In embedded programming

Dynamic allocation is NOT recommended




```
int num = 1;          // data (rw)
int count;            // bss

char *str;            // bss
const int i = 10;     // data (ro)

int main ()
{                                          -----> stack 생성
   int k, a= 1;       // stack

   char *ptr;         // stack
   ptr = malloc(4);   // heap

   static int st_a;      //bss   -> 초기화 안된 변수
   static int st_b = 1;  //data  -> 초기화 된 변수

   c= a+b;            // code (text)
   k = 2;             // code (text)

   func (c, k);       // stack

   return 0;

}                                          -----> stack 소멸

void func (int test, int call)  // stack
{
}
```

Peripheral Register

- **Peripheral register**
  - Peripherals: covers the memory addresses such as GPIO, timers, USART, SPI, and ADC
  - Use **bit manipulation** to change certain bits of a register (not all contents)
  - Require some contents (bit-fields) to be **preserved or masked**

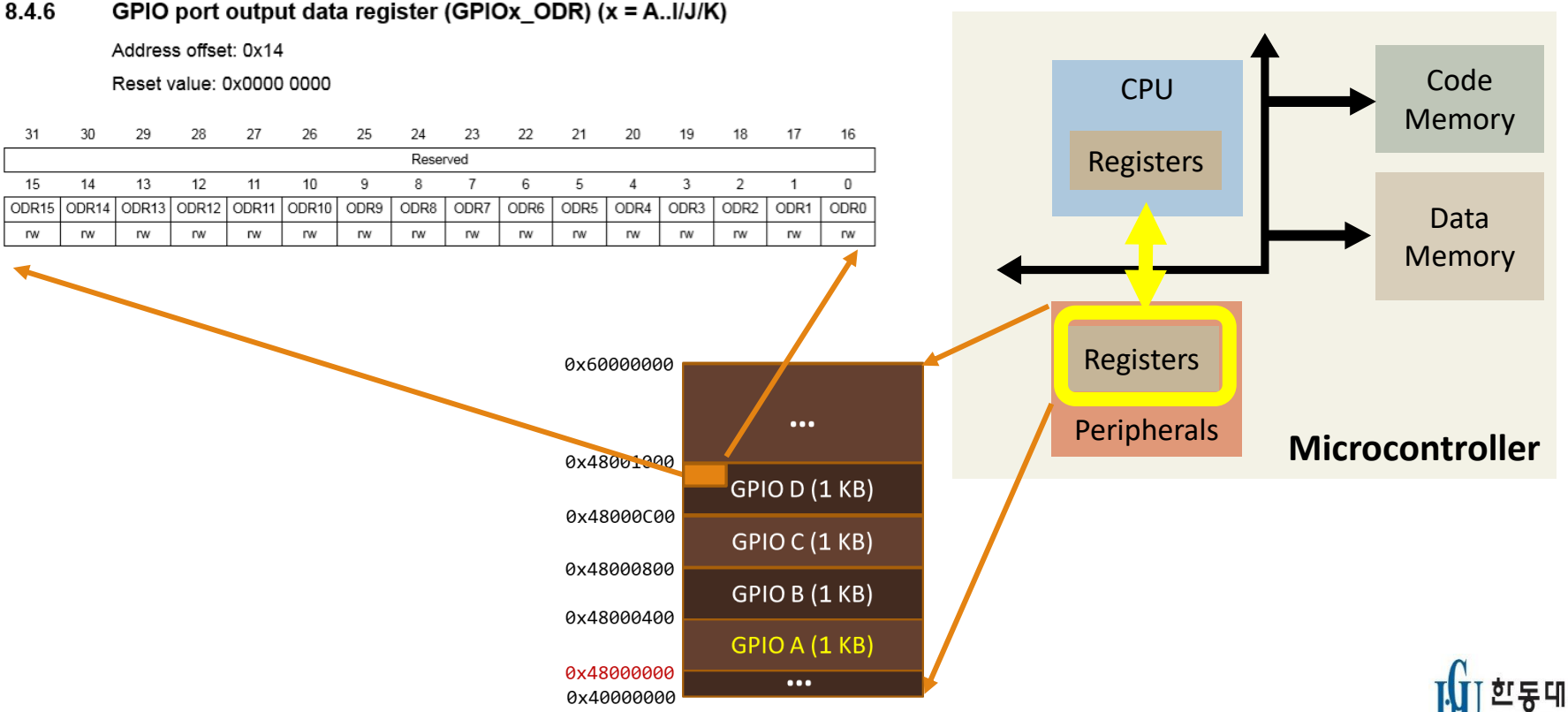
-> we will work on **accessing and controlling** the peripheral registers for I/O

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..I/J/K)

Address offset: 0x14

Reset value: 0x0000 0000

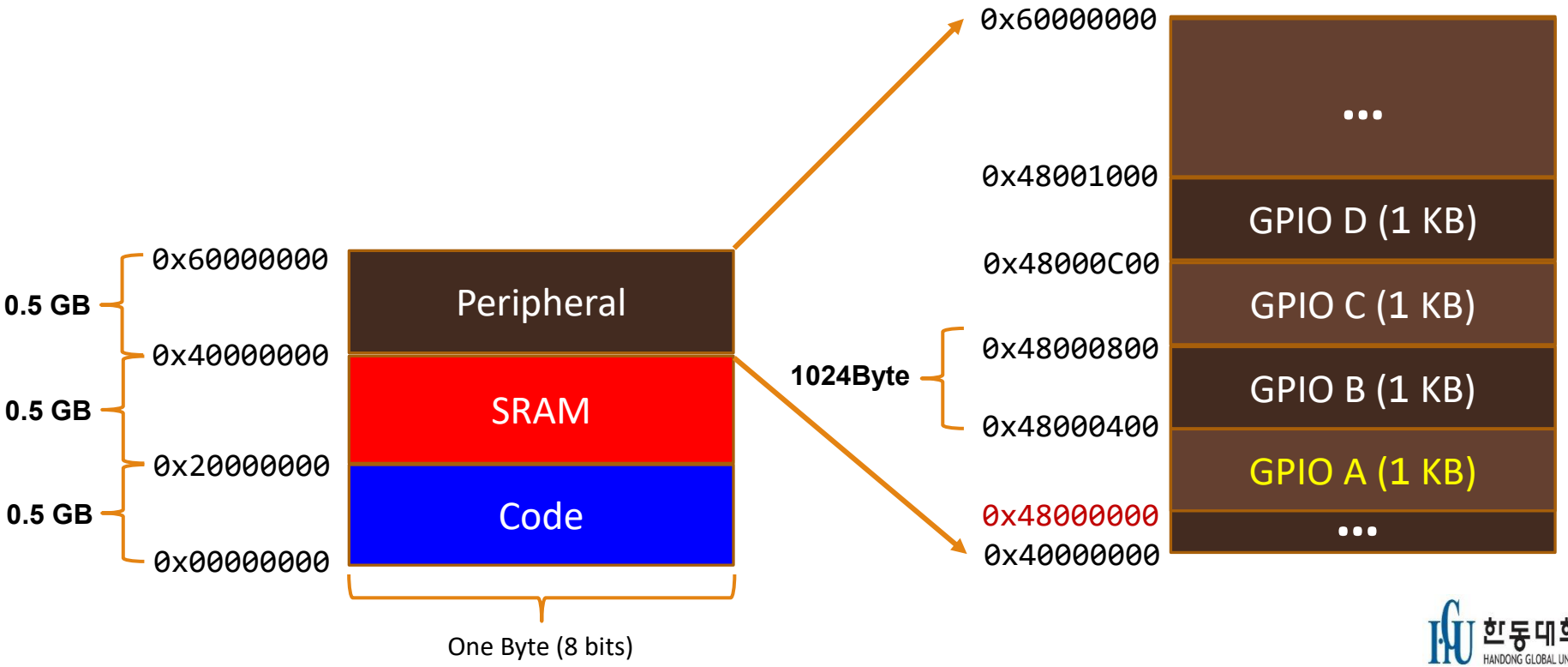
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

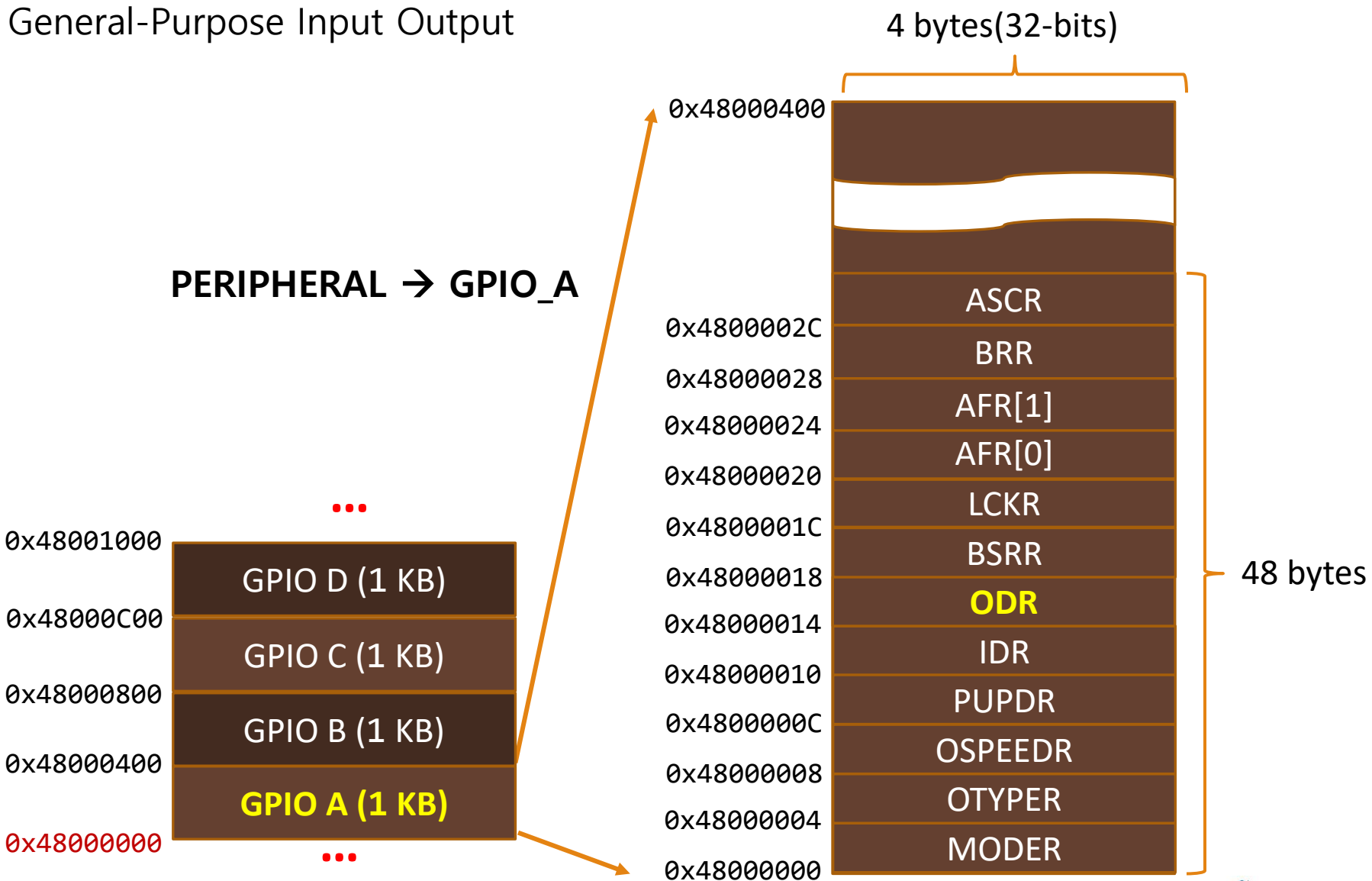
- **Peripheral Register: Memory-mapped I/O**
  - Covers the memory addresses for Peripherals
    - GPIO, timers, USART, SPI, ADC, more
  - GPIO (General-Purpose I/O) pins [0:31] is grouped as Port
    - Port A ~Port I/J/K



한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

- **Peripheral Register: GPIO**
- General-Purpose Input Output



한동대학교 HANDONG GLOBAL UNIVERSITY

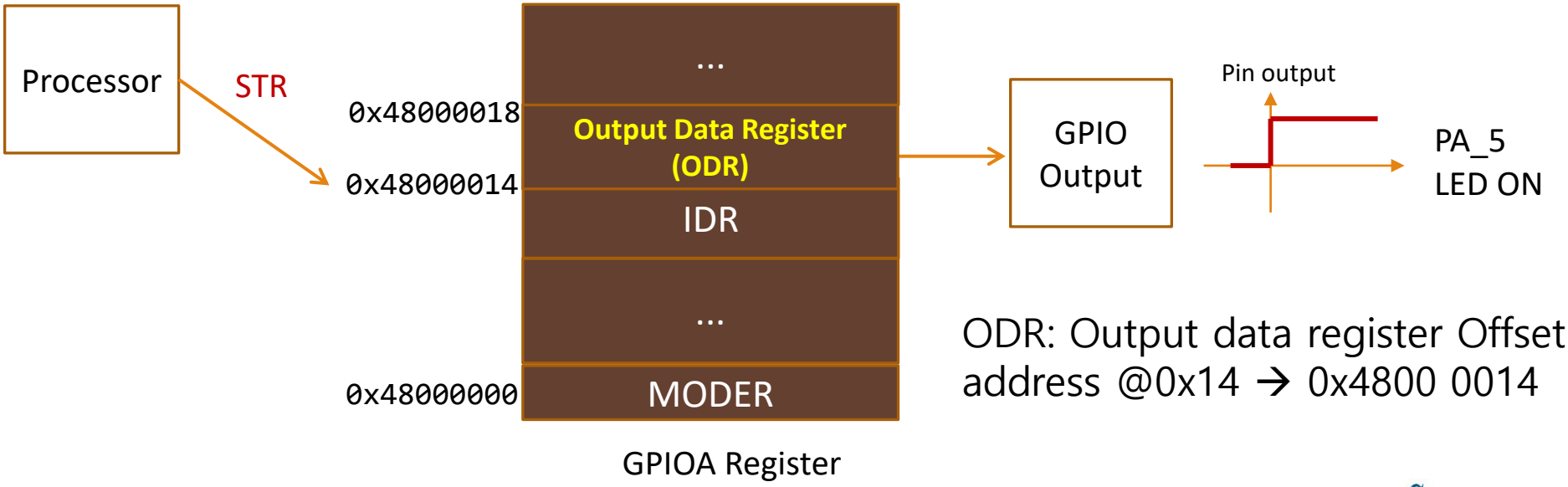
Peripheral Register

- **Peripheral Register: Memory-mapped I/O**
  - Use **bit manipulation** to change certain bits of a register

Example 1: **PortA Pin5** Set HIGH

ODR: Output data register (32bits)

PERIPHERAL→ GPIO_A → ODR : use bitwise to SET as … xxx1 xxxx



GPIOA Register

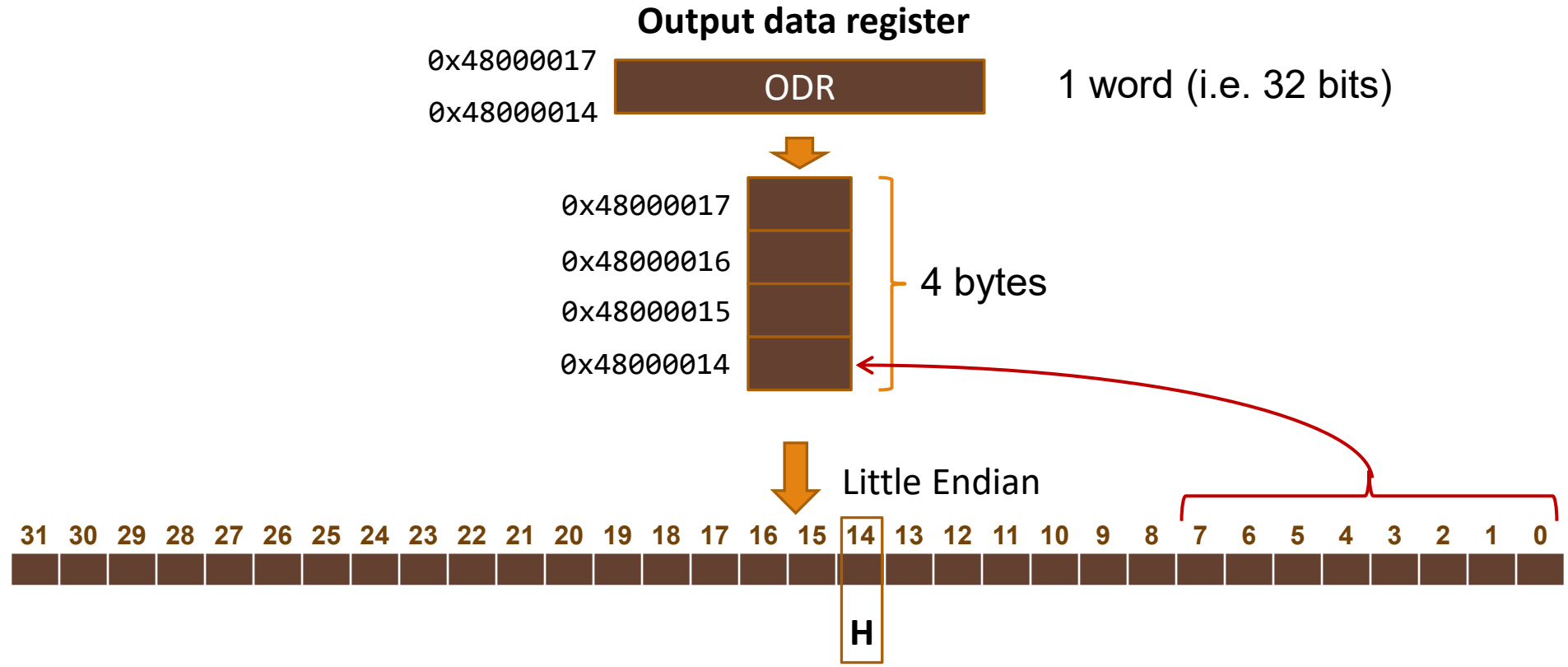
ODR: Output data register Offset address @0x14 → 0x4800 0014

한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

- **Peripheral Register: Memory-mapped I/O**
  - **Example 2: PortA Pin14** Set HIGH

PERIPHERAL→ GPIO_A → ODR :

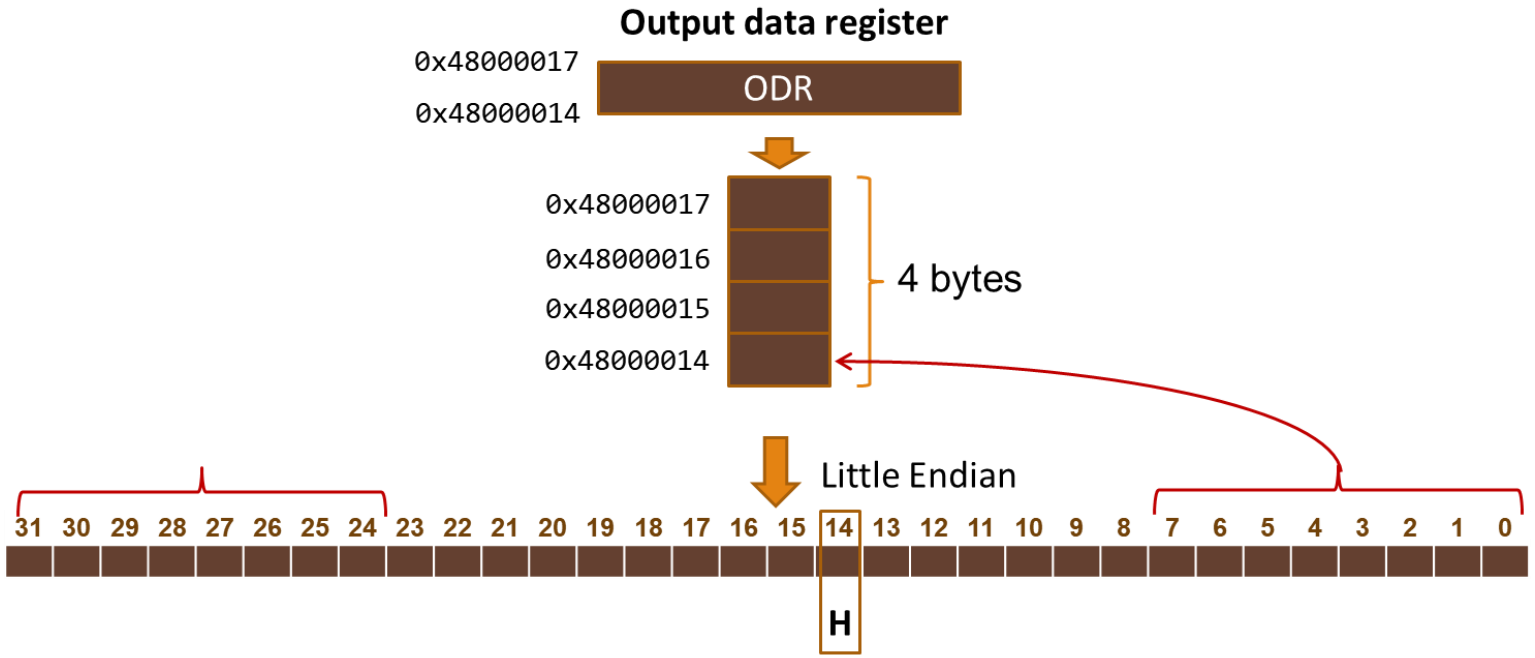


한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

- **Peripheral Register: Memory-mapped I/O**
  - **Example 2: PortA Pin14 Set HIGH**

PERIPHERAL→ GPIO_A → ODR :



Typecasting

```
*((uint32_t *) 0x48000014) |= 1<<14;
```

Write '1' on Bit 14

Dereferencing(accessing) a pointer

Bitwise OR

한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

- **Peripheral Register: Memory-mapped I/O**
  - **Example 2: PortA Pin14** Set HIGH

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..I/J/K)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

H

Output data register

0x48000017

0x48000014

ODR




```
*((uint32_t *) 0x48000014) |= 1UL<<14;
```

Dereferencing a pointer

Bitwise OR

한동대학교 HANDONG GLOBAL UNIVERSITY

Peripheral Register

- **Peripheral Register: Memory-mapped I/O**
  - In C firmware programming : CMSIS defined
  - We will cover more in **"GPIO I/O"**




```
GPIOA->ODR |= 1UL<<14;
```

or

```
(*GPIOA).ODR |= 1UL<<14;
```

```
typedef struct {
  volatile uint32_t MODER;    // Mode register
  volatile uint32_t OTYPER;   // Output type register
  volatile uint32_t OSPEEDR;  // Output speed register
  volatile uint32_t PUPDR;    // Pull-up/pull-down register
  volatile uint32_t IDR;      // Input data register
  volatile uint32_t ODR;      // Output data register
  volatile uint32_t BSRR;     // Bit set/reset register
  volatile uint32_t LCKR;     // Configuration lock register
  volatile uint32_t AFR[2];   // Alternate function registers
  volatile uint32_t BRR;      // Bit Reset register
  volatile uint32_t ASCR;     // Analog switch control register
} GPIO_TypeDef;

// Casting memory address to a pointer
#define GPIOA ((GPIO_TypeDef *) 0x48000000)
```


한동대학교 HANDONG GLOBAL UNIVERSITY

Appendix

- **SRAM (Static RAM)**
  - Contains 4 transistors and 2 resistors or 6 transistors
  - Very fast but expensive due to less memory density per area
  - Used in computers which requires fast memories

  - How to write data
    - "HIGH" on Word Line
    - Put values on Bit Line
    - "low' on Word Line

  - How to read data
    - OFF the Word Line
    - Pre-charge Bit Line
    - "High" on Word Line
    - Bit Line will be "high" or "Low"

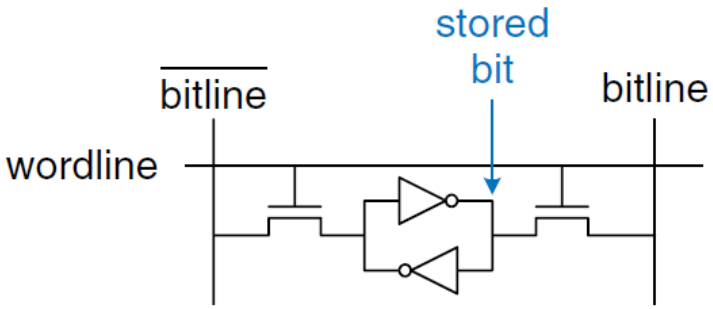


Figure 5.46 SRAM bit cell

- **DRAM (Dynamic RAM)**
  - Contains 1 transistor and 1 capacitor
  - Slower speed than SRAM
    - DDR SDRAM: double edge clock
  - DRAM memories are currently 10 to 100 times slower than processors.
  - Low Cost and High-Power Consumption
  - Need to refresh every few milliseconds
    - the charge on the capacitor gradually leaks away

  - How to write data
    - "high" on Word Line
    - Put values on Bit Line
    - "low' on Word Line

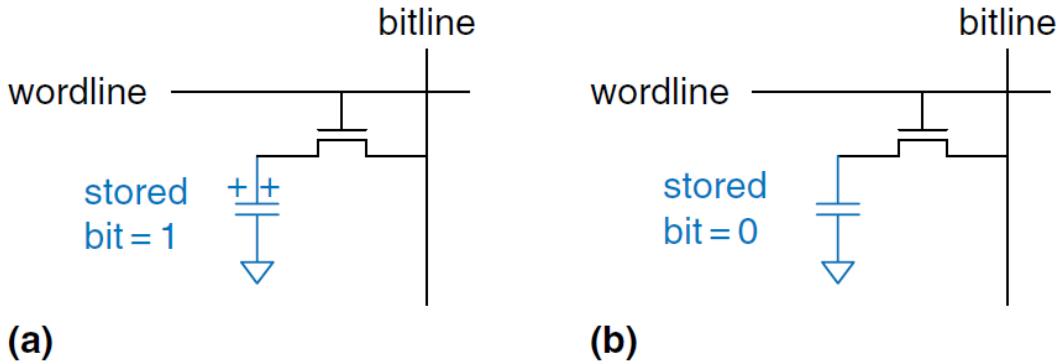


Figure 5.45 DRAM stored values

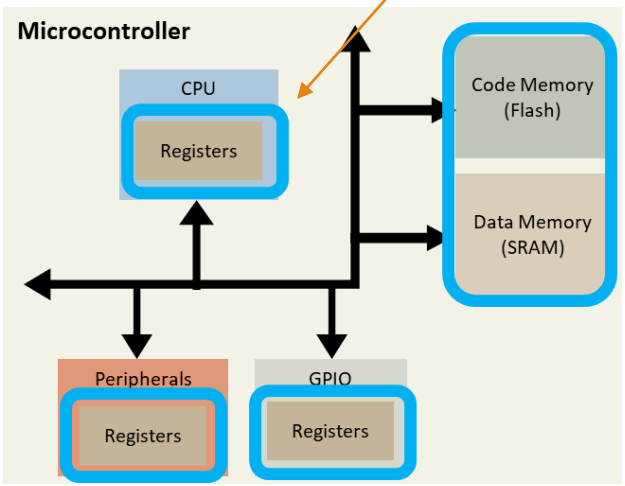
RAM

- Volatile memory comparison:
  - Register, SRAM, DRAM
  - Consider Area, Power, Cost, and Latency
  - One FF can be used for registers (storing a bit)

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

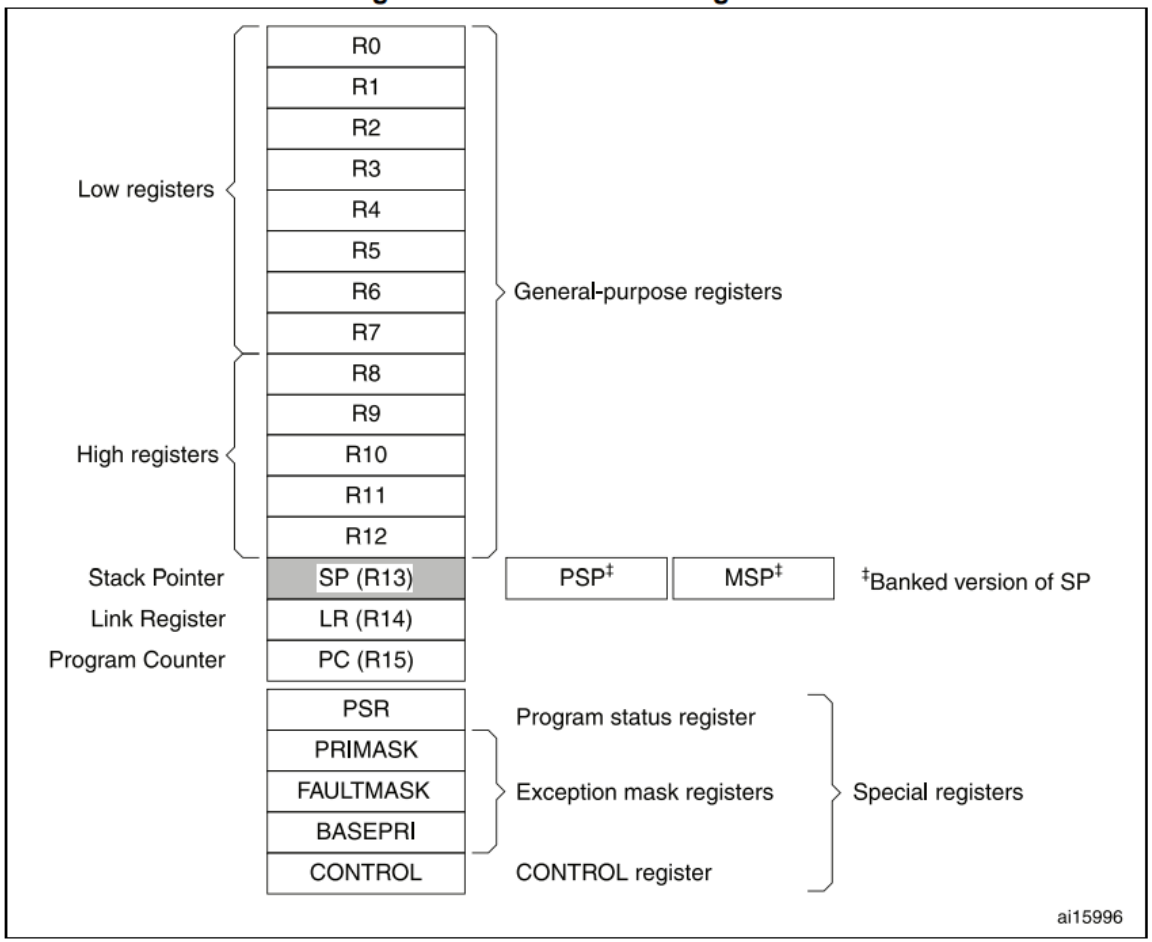
CPU Register

- **Processor Register**
  - Registers are fast memory usually composed of Flip-Flops
  - Used mainly for fast data processing for CPU
  - ARM 32-bit CPU has
    - General Purpose Register
    - Stack Pointer(SP)
    - Link Register(LR)
    - Program Counter(PC)
    - Special Register



Interacting Memory [Coursera]

Figure 2. Processor core registers

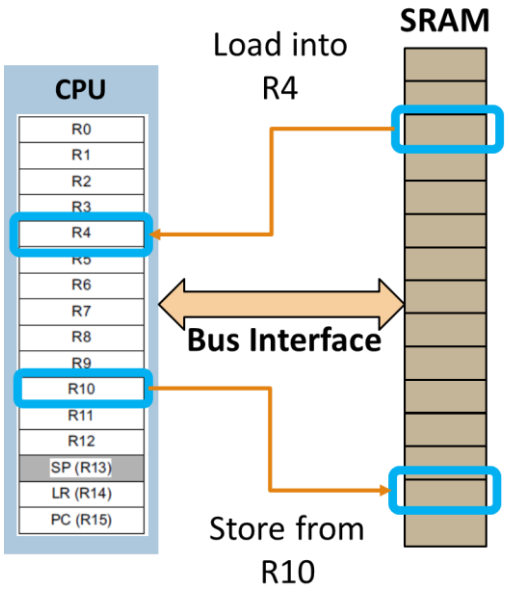


한동대학교 HANDONG GLOBAL UNIVERSITY

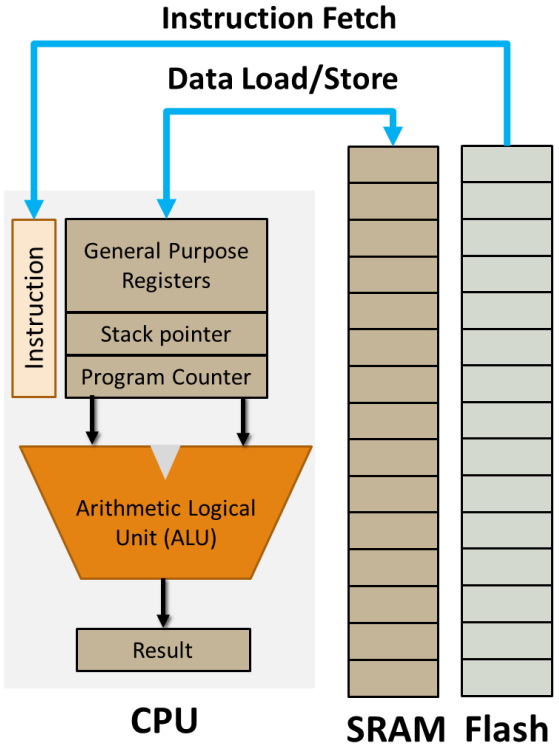
CPU Register

- **Registers are very fast, but expensive**
  - MCU processor register memory is small
  - ARM-Cortex M has only 16 registers (excluding special registers)

- **CPU Register data constantly changes**
  - Data is **loaded** in from memory(SRAM)
  - Results are **stored** back to memory



Instruction are fetched from Flash to CPU



Interacting Memory [3]

한동대학교 HANDONG GLOBAL UNIVERSITY